

Modül 02: R’da Veri Türleri, Veri Yapıları ve Temel Fonksiyonlar

Prof. Dr. Hakan Mehmetcik

2026-09-10

Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler:

- R’daki temel veri türlerini (numeric, character, logical) tanımlayabilecek,
- Basit matematiksel ve mantıksal işlemleri uygulayabilecek,
- Yerleşik fonksiyonların (sum(), mean(), length(), vb.) kullanımını kavrayabilecek,
- Vektör, liste, matris ve data frame gibi basit veri yapıları ile çalışabilecek,
- Bir CSV dosyasını R’a aktarıp inceleyebilecek.

1. Değişkenler ve Veri Türleri

R dilinde, verileri saklamak ve işlemek için değişkenler kullanılır. Değişkenler, belirli bir değeri tutar ve bu değer üzerinde işlemler yapmanıza olanak tanır.

Değişken Tanımlama (<- Operatörü) R’da bir değişken tanımlamak için <- operatörü kullanılır. Bu operatör, sağdaki değeri soldaki değişkene atar.

```
x <- 10    # x değişkenine 10 değerini atar  
y <- "Merhaba"  # y değişkenine bir karakter dizisi atar
```

R'da Veri Türleri

R'da değişkenlerin farklı veri türleri olabilir. R'ın sık kullanılan veri türleri şunlardır:

- **Sayısal (Numeric):** Ondalıklı veya tam sayı olarak saklanan sayılar.

```
a <- 5.7 # Ondalıklı bir sayısal veri  
b <- 10 # Tam sayı olarak bir sayısal veri
```

- **Karakter (Character):** Metin veya harf dizisi olarak saklanan veriler.

```
metin <- "R programlama dili" # Karakter veri türünde bir veri
```

⚠ Değişken adı seçerken dikkat

R, bir değişkene `c`, `data`, `sum`, `mean` gibi **yerleşik fonksiyon adları** vermenizi engellemez. `c <- "metin"` yazmak hata üretmez, ama kodunuzu okuyan (ve birkaç hafta sonra sizin de olacağınız) kişi için ciddi bir kafa karışıklığı kaynağıdır. Kural: **değişken adları fonksiyon adlarıyla çakışmasın.**

- **Mantıksal (Logical):** TRUE veya FALSE değerlerini saklayan değişkenler.

```
durum <- TRUE # Logical
```

Değişkenlerle Matematiksel İşlemler

Sayısal veri türüne sahip değişkenlerde matematiksel işlemler yapabilirsiniz. Örneğin:

```
a <- 5  
b <- 3
```

Toplama

```
toplam <- a + b  
print(toplam) # Çıktı: 8
```

[1] 8

Çıkarma

```
cikarma <- a - b  
print(cikarma) # Çıktı: 2
```

```
[1] 2
```

Çarpma

```
a*b # Çıktı: 15
```

```
[1] 15
```

Bölme

```
bolme <- a / b  
print(bolme) # Çıktı: 1.666667
```

```
[1] 1.666667
```

Üs Alma

```
us <- a^b  
print(us) # Çıktı: 125
```

```
[1] 125
```

2. R'da Temel Fonksiyonlar

R programlama dilinde temel fonksiyonlar, veri analizi ve istatistiksel hesaplamalar için yaygın olarak kullanılır. İşte bazı temel fonksiyonlar ve örnek kullanımları:

R'ın Yerleşik Fonksiyonları Yerleşik fonksiyonlar, R dilinin varsayılan olarak sunduğu işlevlerdir. Örneğin, `mean()` fonksiyonu bir vektörün ortalamasını hesaplar, `sum()` toplamını verir ve `length()` bir vektörün kaç eleman içerdiğini gösterir.

Örnekler:

1. `print()` Bir nesneyi konsola yazdırır.

```
print("Merhaba Dünya")
```

```
[1] "Merhaba Dünya"
```

2. `sum()`: Bir vektörün elemanlarının toplamını hesaplar.

```
toplam <- sum(c(1, 2, 3, 4, 5))
```

3. mean(): Bir vektörün elemanlarının ortalamasını hesaplar.

```
ortalama <- mean(c(1, 2, 3, 4, 5))
```

4. sd(): Bir vektörün standart sapmasını hesaplar.

```
standart_sapma <- sd(c(1, 2, 3, 4, 5))
```

5. length(): Bir vektörün uzunluğunu döndürür.

```
uzunluk <- length(c(1, 2, 3, 4, 5))
```

6. seq(): Belirli bir aralıkta ardışık sayılar oluşturur.

```
dizi <- seq(1, 10, by=2)
```

7. rep(): Bir değeri belirli sayıda tekrar eder

```
tekrar <- rep(1, times=5)
```

8. c(): Vektör oluşturur.

```
vektor <- c(1, 2, 3, 4, 5)
```

9. data.frame(): Veri çerçevesi oluşturur.

```
veri <- data.frame( isim = c("Ali", "Ayşe", "Fatma"), yas = c(25, 30, 35) )
```

10. summary(): Bir nesnenin özet istatistiklerini döndürür.

```
ozet <- summary(c(1, 2, 3, 4, 5))
```

Parametrelerle Fonksiyon Kullanımı

Fonksiyonlar, belirli bir işlevi gerçekleştirmek için parametre alabilir. Parametreler, fonksiyonun nasıl çalışacağını belirler. Örneğin, mean() fonksiyonuna na.rm = TRUE parametresi eklenerek eksik değerleri göz ardı edebilirsiniz.

```
sayilar <- c(10, 20, NA, 40, 50)
ortalama <- mean(sayilar, na.rm = TRUE) # NA değerini göz ardı ederek ortalama hesaplar
```

Fonksiyon Oluşturma

R dilinde kendi fonksiyonlarınızı oluşturmak oldukça basittir. Fonksiyonlar, belirli bir işlevi gerçekleştirmek için bir dizi komut içerir ve gerektiğinde parametreler alabilir. Fonksiyonlar, `function` anahtar kelimesi kullanılarak tanımlanır.

Fonksiyon Oluşturma Bir fonksiyon oluşturmak için aşağıdaki adımları izleyebilirsiniz:

- `function` anahtar kelimesini kullanarak fonksiyonunuzu tanımlayın.
- Fonksiyonun alacağı parametreleri parantez içinde belirtin.
- Fonksiyonun gövdesinde, fonksiyonun gerçekleştireceği işlemleri yazın.
- `return()` fonksiyonunu kullanarak fonksiyonun döndüreceği değeri belirtin (isteğe bağlı).

```
# Toplama fonksiyonu oluşturma
topla <- function(a, b) {
  sonuc <- a + b
  return(sonuc)
}

# Fonksiyonu çağırma
toplama <- topla(5, 3)
print(toplama) # 8
```

```
[1] 8
```

R Fonksiyonları İçin Yardım Alma (? ve help() Komutları)

Bir R fonksiyonu hakkında bilgi almak için `?` veya `help()` komutlarını kullanabilirsiniz. Bu komutlar, ilgili fonksiyonun nasıl kullanılacağı ve parametrelerinin ne olduğu hakkında detaylı bilgi verir.

Örnekler:

```
?mean      # mean() fonksiyonu hakkında yardım alır
help(sum)  # sum() fonksiyonu hakkında yardım alır
```

Bu komutlar, fonksiyonun açıklamasını, hangi argümanları kabul ettiğini ve nasıl çalıştırılacağını gösteren bir dokümantasyon penceresi açar.

Belgeleme ve Fonksiyonların Anlamı

R'ın tüm fonksiyonlarının nasıl çalıştığını, örnekler ve açıklamalarla birlikte görebilirsiniz. R'da her fonksiyonun geniş bir açıklaması bulunur ve bu belgeleme, fonksiyonların nasıl kullanılacağını öğrenmenin etkili bir yoludur.

Örnek:

```
help(mean) # Ortalama hesaplayan mean() fonksiyonunu anlamak için yardım alabiliriz
```

3. Basit Veri Yapıları

R dilinde, veriler çeşitli veri yapıları kullanılarak organize edilir. Temel veri yapıları, vektörler, listeler ve matrislerdir.

Vektörler

R dilinde en temel veri yapılarından biri vektörlerdir. Vektörler, aynı veri türüne sahip bir dizi elemanı saklar. Vektör oluşturmak için `c()` fonksiyonu kullanılır.

```
# Sayısal vektör oluşturma
numeric_vector <- c(1, 2, 3, 4, 5)

# Karakter vektör oluşturma
character_vector <- c("a", "b", "c", "d")

# Mantıksal vektör oluşturma
logical_vector <- c(TRUE, FALSE, TRUE)

# Vektör elemanlarına erişim
print(numeric_vector[1]) # 1
```

```
[1] 1
```

```
print(character_vector[3]) # "c"
```

```
[1] "c"
```

```
# Vektör elemanlarını güncelleme
numeric_vector[2] <- 10

# Vektör toplama
sum_vector <- numeric_vector + c(1, 1, 1, 1, 1)

# Vektör elemanlarının toplamı
total_sum <- sum(numeric_vector)

# Güncellenmiş vektörleri yazdırma
print(numeric_vector)
```

```
[1] 1 10 3 4 5
```

```
print(sum_vector)
```

```
[1] 2 11 4 5 6
```

```
print(total_sum)
```

```
[1] 23
```

Mantıksal İndeksleme

Yukarıda vektör elemanlarına **konumlarına** göre eriştik: `vektor[2]` “ikinci elemanı ver” demektir. Ama gerçek veri analizinde soru neredeyse hiç böyle sorulmaz. “Beşinci ülkenin nüfusu nedir?” diye sormayız; “nüfusu 50 milyondan **büyük olan** ülkeler hangileridir?” diye sorarız.

Bunun için R’da köşeli parantezin içine bir konum numarası değil, bir **mantıksal ifade** yazarız.

Adım 1: Karşılaştırma bir mantıksal vektör üretir

```
sayilar <- c(4, 7, 2, 9, 5)
sayilar > 5
```

```
[1] FALSE TRUE FALSE TRUE FALSE
```

Dikkat edin: çıktı bir sayı değil, **sayılar** ile **aynı uzunlukta** bir TRUE/FALSE vektörüdür. Her eleman için “bu koşulu sağlıyor mu?” sorusunun cevabıdır.

Adım 2: Bu maskeyi indeks olarak kullanınız

```
sayilar[sayilar > 5]
```

```
[1] 7 9
```

R, TRUE olan konumlardaki elemanları tutar, FALSE olanları atar. Bu işleme **mantıksal indekсле**me (İng. *logical subsetting*) ya da **maskeleme** denir.

Koşulları 1. oturumda gördüğünüz & (ve), | (veya), ! (değil) operatörleriyle birleştirebilirsiniz:

```
sayilar[sayilar > 3 & sayilar < 9] # 3'ten büyük VE 9'dan küçük
```

```
[1] 4 7 5
```

```
sayilar[!(sayilar == 2)] # 2'ye eşit OLMAYANLAR
```

```
[1] 4 7 9 5
```

💡 Sık yapılan hata: = ile == karışıklığı

sayilar[sayilar = 5] hata verir. Eşitliği sınamak için daima çift eşittir (==) kullanın; tek eşittir (=) atama içindir.

Listeler

Listeler, farklı türdeki verileri bir arada saklayabilen veri yapılarıdır. Bir liste içinde sayılar, karakterler, mantıksal değerler ve hatta başka vektörler bulunabilir.

```
# Liste oluşturma
my_list <- list(number = 5, greeting = "Hello", flag = TRUE, numbers = c(1, 2, 3))

# Liste elemanlarına erişim
print(my_list$number) # 5
```



```
[1] 5
```

```
print(my_list$greeting) # "Hello"
```

```
[1] "Hello"
```

```
print(my_list$flag) # TRUE
```

```
[1] TRUE
```

```
print(my_list$numbers) # c(1, 2, 3)
```

```
[1] 1 2 3
```

```
# Liste elemanlarını güncelleme
my_list$number <- 10
my_list$greeting <- "Hi"

# Listeye yeni eleman ekleme
my_list$new_element <- "New Element"

# Liste elemanlarını silme
my_list$new_element <- NULL

# Güncellenmiş listeyi yazdırma
print(my_list)
```

```
$number
```

```
[1] 10
```

```
$greeting
```

```
[1] "Hi"
```

```
$flag
```

```
[1] TRUE
```

```
$numbers
```

```
[1] 1 2 3
```

Matrisler

Matrisler, satırlar ve sütunlar şeklinde organize edilmiş sayısal veri yapılarıdır. Tüm elemanlar aynı türde olmalıdır.

```
# 2x3 boyutunda bir matris oluşturma
my_matrix <- matrix(1:6, nrow = 2, ncol = 3)

# Matris elemanlarına erişim
print(my_matrix[1, 2]) # 3
```

```
[1] 3
```

```
print(my_matrix[2, ]) # c(4, 5, 6)
```

```
[1] 2 4 6
```

```
print(my_matrix[, 3]) # c(5, 6)
```

```
[1] 5 6
```

```
# Matris elemanlarını güncelleme
my_matrix[1, 2] <- 10

# Matris toplama
matrix1 <- matrix(1:4, nrow = 2)
matrix2 <- matrix(5:8, nrow = 2)
sum_matrix <- matrix1 + matrix2

# Matris çarpma
product_matrix <- matrix1 %*% matrix2

# Güncellenmiş matrisleri yazdırma
print(my_matrix)
```

```
      [,1] [,2] [,3]
[1,]    1  10    5
[2,]    2   4    6
```

```
print(sum_matrix)
```

```
      [,1] [,2]  
[1,]     6  10  
[2,]     8  12
```

```
print(product_matrix)
```

```
      [,1] [,2]  
[1,]    23  31  
[2,]    34  46
```

Data Frameler

Data frameler, R dilinde kullanılan ve farklı veri türlerine sahip sütunları içerebilen iki boyutlu veri yapılarıdır. Data frameler, genellikle tablo şeklinde verileri saklamak için kullanılır ve her sütun bir vektör olarak temsil edilir. Data frameler, `data.frame()` fonksiyonu kullanılarak oluşturulur.

```
# Data frame oluşturma  
my_data_frame <- data.frame(  
  numbers = c(1, 2, 3, 4, 5),  
  letters = c("a", "b", "c", "d", "e"),  
  logicals = c(TRUE, FALSE, TRUE, FALSE, TRUE)  
)
```

```
# Data frame elemanlarına erişim  
print(my_data_frame[1, 2]) # "a"
```

```
[1] "a"
```

```
print(my_data_frame[2, ]) # tek satırlık bir data frame (vektör değil!)
```

```
  numbers letters logicals  
2       2       b    FALSE
```

```
print(my_data_frame[, 3]) # c(TRUE, FALSE, TRUE, FALSE, TRUE)
```

```
[1] TRUE FALSE TRUE FALSE TRUE
```

```
print(my_data_frame$numbers) # c(1, 2, 3, 4, 5)
```

```
[1] 1 2 3 4 5
```

```
# Data frame elemanlarını güncelleme
my_data_frame[1, 2] <- "z"
my_data_frame$numbers[2] <- 10

# Mantıksal indekslemeyi satır filtrelemek için kullanma
# Virgülden ÖNCESİ satırları, SONRASI sütunları seçer:
my_data_frame[my_data_frame$numbers > 2, ]
```

	numbers	letters	logicals
2	10	b	FALSE
3	3	c	TRUE
4	4	d	FALSE
5	5	e	TRUE

```
# Yeni bir sütun ekleme
my_data_frame$new_column <- c(10, 20, 30, 40, 50)

# Satır ekleme
new_row <- data.frame(numbers = 6, letters = "f", logicals = FALSE, new_column = 60)
my_data_frame <- rbind(my_data_frame, new_row)

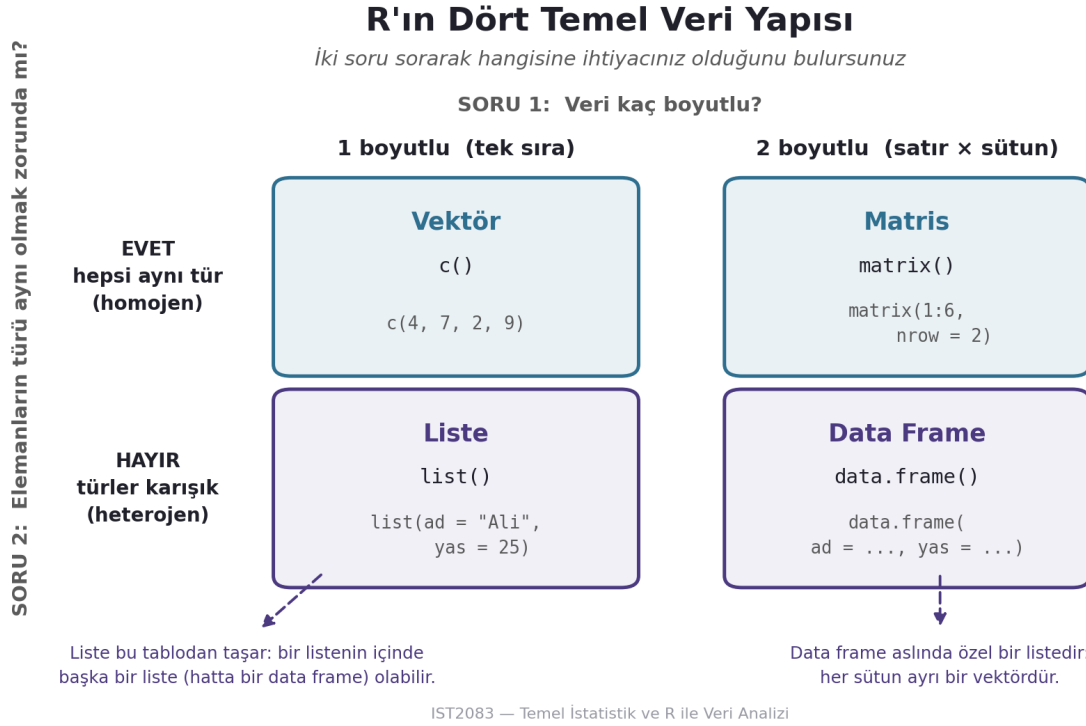
# Sütun ekleme
new_column <- c(100, 200, 300, 400, 500, 600)
my_data_frame <- cbind(my_data_frame, new_column)

# Güncellenmiş data frame'i yazdırma
print(my_data_frame)
```

	numbers	letters	logicals	new_column	new_column
1	1	z	TRUE	10	100
2	10	b	FALSE	20	200
3	3	c	TRUE	30	300
4	4	d	FALSE	40	400
5	5	e	TRUE	50	500
6	6	f	FALSE	60	600

Dördünü Birlikte Görmek

Bu dört yapıyı ezberlemenize gerek yok. Elinizdeki veriye **iki soru** sorarsanız hangisine ihtiyacınız olduğunu kendiniz bulursunuz: veri kaç boyutlu, ve elemanların türü aynı olmak zorunda mı?



Figür 1: R'ın dört temel veri yapısının boyut ve tür-homojenliği eksenlerindeki konumu

Şemanın altındaki iki not önemlidir. **Liste bu tablodan taşar:** bir listenin elemanı yine bir liste olabilir, bu yüzden aslında iki boyutlu bir tabloya tam oturmaz — R'ın en esnek yapısıdır. **Data frame ise aslında özel bir listedir:** her sütunu bir vektör olan, tüm sütunları eşit uzunlukta olmak zorunda olan bir liste. Bu yüzden hem `df$sutun` (liste sözdizimi) hem `df[1, 2]` (tablo sözdizimi) çalışır.

i 3. oturuma köprü

`my_data_frame[my_data_frame$numbers > 2, ]` yazımı doğru ama hantaldır: veri çerçevesinin adını iki kez tekrarlamak zorunda kalırsınız. 3. haftada `dplyr` paketini öğrendiğinizde aynı işi `filter(my_data_frame, numbers > 2)` biçiminde yazacaksınız. `dplyr`'ın yaptığı sihir bu değildir — **taban R'da zaten var olan bu mantığı** okunabilir

hale getirmesidir. Bu yüzden önce burayı öğreniyoruz.

4. Veri İçerik Aktarma ve Kaydetme

Veri içerik aktarma ve temel manipülasyon, veri analizi sürecinin önemli adımlarıdır. R dilinde veri içerik aktarma ve temel manipülasyon işlemleri için çeşitli fonksiyonlar ve paketler kullanılır.

Veri İçerik Aktarma

R dilinde veri içerik aktarmak için `read.csv()`, `read.table()`, `read_excel()` gibi fonksiyonlar kullanılır.

```
# CSV dosyasını içerik aktarma (doğrudan bir internet adresinden)
titanic <- read.csv("https://raw.githubusercontent.com/datasciencedojo/datasets/refs/heads/main/titanic.csv")

# readxl paketini yükleme
# install.packages("readxl")
library(readxl)
library(here)
```

`here()` starts at `/Users/kobain/Dersler/IST2083`

```
# Excel dosyasını içerik aktarma (depo içindeki data/ klasöründen)
ornek <- read_excel(here("data", "sample_excel.xlsx"))
```

Veri Kaydetme

R dilinde veri kaydetmek için çeşitli fonksiyonlar kullanabilirsiniz. En yaygın kullanılan dosya formatları arasında CSV, Excel ve RDS dosyaları bulunur.

CSV Dosyasına Veri Kaydetme Veri çerçevesini CSV dosyasına kaydetmek için `write.csv()` fonksiyonunu kullanabilirsiniz:

```
# Veri çerçevesini CSV dosyasına kaydetme
# write.csv(data, "path/to/your/file.csv", row.names = FALSE)
```

Excel Dosyasına Veri Kaydetme Excel dosyasına veri kaydetmek için writexl paketini kullanabilirsiniz:

```
# writexl paketini yükleme
# install.packages("writexl")
# library(writexl)

# Veri çerçevesini Excel dosyasına kaydetme
# write_xlsx(data, "path/to/your/file.xlsx")
```

Veri Yapısını İnceleme

Veri inceleme, veri analizi sürecinin önemli bir parçasıdır. R dilinde veri incelemek için çeşitli fonksiyonlar ve paketler kullanabilirsiniz. Bu işlemler arasında veri yapısını kontrol etme, özet istatistikler oluşturma ve veri görselleştirme gibi işlemler bulunur. Veri yapısını incelemek için `str()`, `summary()`, `head()`, ve `tail()` fonksiyonlarını kullanabilirsiniz.

```
# Veri yapısını kontrol etme: kaç satır, kaç sütun, her sütun hangi türde?
str(titanic)
```

```
'data.frame':  891 obs. of  12 variables:
 $ PassengerId: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Survived   : int  0 1 1 1 0 0 0 0 1 1 ...
 $ Pclass     : int  3 1 3 1 3 3 1 3 3 2 ...
 $ Name       : chr  "Braund, Mr. Owen Harris" "Cumings, Mrs. John Bradley (Florence Briggs T
 $ Sex        : chr  "male" "female" "female" "female" ...
 $ Age        : num  22 38 26 35 35 NA 54 2 27 14 ...
 $ SibSp      : int  1 1 0 1 0 0 0 3 0 1 ...
 $ Parch      : int  0 0 0 0 0 0 0 1 2 0 ...
 $ Ticket     : chr  "A/5 21171" "PC 17599" "STON/O2. 3101282" "113803" ...
 $ Fare       : num  7.25 71.28 7.92 53.1 8.05 ...
 $ Cabin      : chr  "" "C85" "" "C123" ...
 $ Embarked   : chr  "S" "C" "S" "S" ...
```

```
# Sayısal sütunlar için özet istatistikler
summary(titanic)
```

PassengerId	Survived	Pclass	Name
Min. : 1.0	Min. :0.0000	Min. :1.000	Length :891
1st Qu.:223.5	1st Qu.:0.0000	1st Qu.:2.000	N.unique :891

```

Median :446.0   Median :0.0000   Median :3.000   N.blank : 0
Mean    :446.0   Mean    :0.3838   Mean    :2.309   Min.nchar: 12
3rd Qu.:668.5   3rd Qu.:1.0000   3rd Qu.:3.000   Max.nchar: 82
Max.    :891.0   Max.    :1.0000   Max.    :3.000

```

```

Sex      Age      SibSp      Parch
Length   :891   Min.     : 0.42   Min.     :0.000   Min.     :0.0000
N.unique : 2   1st Qu.:20.12   1st Qu.:0.000   1st Qu.:0.0000
N.blank  : 0   Median :28.00   Median :0.000   Median :0.0000
Min.nchar: 4   Mean    :29.70   Mean    :0.523   Mean    :0.3816
Max.nchar: 6   3rd Qu.:38.00   3rd Qu.:1.000   3rd Qu.:0.0000
Max.     :80.00   Max.     :8.000   Max.     :6.0000
NAs      :177

```

```

Ticket      Fare      Cabin      Embarked
Length      :891   Min.     : 0.00   Length    :891   Length    :891
N.unique    :681   1st Qu.: 7.91   N.unique  :148   N.unique  : 4
N.blank     : 0   Median :14.45   N.blank   :687   N.blank   : 2
Min.nchar   : 3   Mean    :32.20   Min.nchar : 0    Min.nchar : 0
Max.nchar   :18   3rd Qu.:31.00   Max.nchar :15    Max.nchar : 1
Max.        :512.33

```

```

# İlk birkaç satırı görüntüleme
head(titanic)

```

PassengerId	Survived	Class	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.2500	S	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C	
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2 3101282	27.9250	S	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1000	C	
5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.0500	S	
6	0	3	Moran, Mr. James	male	NA	0	0	330877	8.4583	Q	


```
# Son birkaç satırı görüntüleme
tail(titanic)
```

	PassengerId	Survived	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
886	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39	0	5	382652	29.125	Q
887	887	0	2	Montvila, Rev. Juozas	male	27	0	0	211536	13.000	S
888	888	1	1	Graham, Miss. Margaret Edith	female	19	0	0	112053	30.000	B42 S
889	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NA	1	2	W./C. 6607	23.450	S
890	890	1	1	Behr, Mr. Karl Howell	male	26	0	0	111369	30.000	C148 C
891	891	0	3	Dooley, Mr. Patrick	male	32	0	0	370376	7.750	Q

i Not

Ek Kaynak olarak RStudio Cheat Sheet: Base R'a bakmayı unutma!